Red Hat OpenShift AI Self-Managed 3.4

Working with MLflow

Work with MLflow from Red Hat OpenShift AI Self-Managed

Last Updated: 2026-06-16

Red Hat OpenShift AI Self-Managed 3.4 Working with MLflow

Work with MLflow from Red Hat OpenShift AI Self-Managed

Legal Notice

Abstract

As a data scientist or platform engineer, you can use the MLflow operator in OpenShift AI.

Table of Contents

CHAPTER 1. ABOUT MLFLOW

Red Hat OpenShift AI deploys a single shared MLflow instance through the MLflow operator. With this deployment, you can use MLflow workspaces and Kubernetes-backed authorization. For more information, see MLflow Workspaces.

A project is a namespace that has its own set of objects, policies, constraints, and service accounts. Each Red Hat OpenShift Container Platform project maps to an MLflow workspace in a one-to-one relationship. MLflow provides logical isolation of experiments, runs, registered models, prompts, datasets, and traces per workspace.

OpenShift AI manages the workspace lifecycle outside of MLflow. The MLflow API does not create, update, or delete workspaces. Instead, you manage workspaces through the OpenShift AI dashboard or the Red Hat OpenShift command-line interface (CLI). When you manage projects through these tools, the corresponding MLflow workspace becomes available automatically.

Every MLflow API request relies on Kubernetes role-based access control (RBAC) for authorization. The MLflow server submits a SelfSubjectAccessReview against pseudo-resources in the mlflow.kubeflow.org API group, using the caller's bearer token and the target project namespace. Users who already have the Red Hat OpenShift admin, edit, or view roles on a project automatically receive the corresponding MLflow permissions.

CHAPTER 2. INSTALL AND CONFIGURE MLFLOW

To track machine learning experiments in OpenShift AI, use the Red Hat OpenShift AI Operator to deploy a cluster-scoped **mlflow** instance.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift Container Platform
- You have permission to patch the **DataScienceCluster** resource and apply **MLflow** custom resources.
- For production-oriented deployments, you have created the required secrets for database credentials and S3-compatible storage.

Procedure

The MLflow resource is cluster scoped. Therefore, you can install only one instance of **mlflow** in the cluster. The Red Hat OpenShift AI Operator creates the MLflow resource in the redhat-ods-applications namespace.

1. Log in to the OpenShift cluster by using the CLI.
2. Enable the MLflow Operator component by patching the **DataScienceCluster** object:

```
$ oc patch datasciencecluster default-dsc \
  --type=merge \
  -p {"spec":{"components":{"mlflowoperator":{"managementState":"Managed"}}}}
```

3. Create an **MLflow** custom resource (CR). Choose the configuration that matches your environment:

   Minimal development or test deployment

   Uses SQLite for the backend store and a persistent volume claim (PVC) for file-based artifact storage.

```
apiVersion: mlflow.opendatahub.io/v1
kind: MLflow
metadata:
  name: mlflow
  namespace: redhat-ods-applications
spec:
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 10Gi
  backendStoreUri: "sqlite:////mlflow/mlflow.db"
  artifactsDestination: "file:///mlflow/artifacts"
  serveArtifacts: true
```

   Production-oriented deployment

Uses PostgreSQL for metadata and S3-compatible object storage for artifacts.

```
apiVersion: mlflow.opendatahub.io/v1
kind: MLflow
metadata:
  name: mlflow
  namespace: redhat-ods-applications
spec:
  replicas: 2
  backendStoreUriFrom:
    name: mlflow-db-credentials
    key: backend-store-uri
  artifactsDestination: "s3://my-mlflow-bucket/artifacts"
  serveArtifacts: true
  envFrom:
    - secretRef:
        name: aws_access_key_id=<your_aws_access_key>
              aws_secret_access_key=<your_aws_secret_access_key>
              aws_session_token=<your_aws_session_token>
```

Table 2.1. MLflow Deployment and Artifact Configuration

Parameter or Feature	Configuration Details
replicas	Acts as an important scaling knob specifically for production-oriented deployments.
registryStoreUri	Defaults to the value of **backendStoreUri** if not specified. In practice, the same PostgreSQL database is typically utilized for both.
serveArtifacts	When enabled, clients log and retrieve artifacts through the MLflow server REST API via the **mlflow-artifacts:/** proxy URI scheme. If **defaultArtifactRoot** is not specified while this is enabled, it automatically defaults to **mlflow-artifacts:/**.
defaultArtifactRoot	Defines the directory for storing artifacts for each new experiment. Do not set this to a direct-storage URI (e.g., **s3://**) when **serveArtifacts** is enabled; doing so causes clients to bypass the proxy and attempt direct storage access.
Artifact Storage Path	If no per-project override is configured, artifacts are stored under **<default_artifact_root>/workspaces/<project_name>**. Note that file-based artifact storage requires **serveArtifacts: true**.

4. In the OpenShift AI dashboard navigation bar, click the **Applications** menu.
5. Confirm that the **MLflow UI** is displayed in the list.
6. Open the UI, select your workspace, and verify that you can compare runs and visualize metrics.
7. Alternatively, click **Start Demo**, which loads demo data to compare runs and visualize metrics.

Additional resources

- To view all available configuration options for the MLflow custom resource, run the **oc explain mlflow.spec** command.

2.1. MLFLOW AND MLFLOWCONFIG CONFIGURATION PARAMETERS

The MLflow deployment on Red Hat OpenShift AI relies on two custom resources: **MLflow** and **MLflowConfig**. These resources configure metadata storage, artifact destinations, and project-specific overrides.

The **MLflow** resource is cluster-scoped and must be named **mlflow**. This resource defines the global state of the MLflow deployment, including the database backend, artifact storage, replica count, and TLS configuration.

Table 2.2. Key fields in the MLflow resource

Field	Description
spec.replicas	The number of MLflow pods. The default value is 1. High-availability deployments require remote storage because PVC-backed storage does not support multiple concurrent writers.
spec.backendStoreUri	The tracking metadata database URI. Supported values include **sqlite://** for development and **postgresql://** for production. For production usage, use **spec.backendStoreUriFrom** to avoid storing database credentials in the MLflow CR.
spec.backendStoreUriFrom	A secret reference for the backend database URI. This field is used when the URI contains credentials.
mlflow.spec.resources	Specifies the compute resources for the MLflow container.

The **MLflowConfig** resource is namespace-scoped and must be named **mlflow**. Project owners can use this resource to override default artifact storage settings for their specific projects.

Table 2.3. Key fields in the MLflowConfig resource

Field	Description
spec.artifactRootSecret	The name of a secret in the project that contains S3-compatible connection credentials and bucket information. The value must be **mlflow-artifact-connection**.
spec.artifactRootPath	An optional relative path appended to the bucket root from the secret. For example, if the bucket is **my-bucket** and the path is **mlflow-artifacts**, the resolved artifact root is **s3://my-bucket/mlflow-artifacts**.

- To view all available configuration fields for a resource, use the **oc explain <resource>.spec** command.

2.2. AGGREGATE CLUSTER ROLES

MLflow uses aggregate cluster roles to control access to MLflow resources in Red Hat OpenShift AI. These roles determine which MLflow operations a user can perform based on their existing role bindings.

IMPORTANT

Standard bindings for **view**, **edit**, and **admin** roles do not grant identical access to all MLflow resources. While **mlflow** resources are cluster-scoped, **mlflowconfig** resources are namespace-scoped. Effective access depends on whether permissions are granted at the cluster level or the namespace level.

The **mlflow-view** ClusterRole aggregates into the standard OpenShift AI **view**, **edit**, and **admin** roles. User access depends on the scope of the binding set at the cluster level.

This role grants read-only access to the following MLflow resources:

- **get**, **list**, and **watch** permissions for the **mlflow** and **mlflowconfigs** Custom Resource Definitions (CRDs)
- **get** and **list** permissions for MLflow pseudo-resources, such as **datasets**, **experiments**, and **registeredmodels**
- **get** permissions for MLflow status

The **mlflow-edit** ClusterRole aggregates into the standard OpenShift AI **edit** and **admin** roles. User access depends on the scope of the binding set at the cluster level.

This role grants write access to the following MLflow resources:

- **create**, **read**, **update**, and **delete** permissions for the **mlflow** CRD
- **create**, **delete**, **patch**, and **update** permissions for the **mlflowconfigs** CRD
- **create**, **update**, and **delete** permissions for MLflow pseudo-resources
- **update** permissions for **finalizers**

The **mlflow-integration** ClusterRole grants service accounts data-plane access to MLflow without requiring full edit or delete privileges. When you bind this role with a RoleBinding, it provides namespace-scoped access. This ClusterRole does not grant **delete** or **access** to the control-plane resources.

This role is intended for integration-focused tasks and grants the following access:

- **get**, **list**, **create**, and **update** permissions for the **experiments**, **datasets**, and **registeredmodels** authorization-plugin pseudo-resources
- Read access to MLflow resources and statuses
- Permissions to interact with the MLflow tracking API and model registry

2.3. RBAC MODEL FOR MLFLOW API USAGE

The MLflow server authorizes every API request by using a Kubernetes **SelfSubjectAccessReview**. The server uses the caller's bearer token to determine if the token has permission to perform a specific verb on an MLflow pseudo-resource within the target project namespace.

The authorization checks use the **mlflow.kubeflow.org** API group. These *pseudo-resources* are used solely for role-based access control (RBAC) policy evaluation. Pseudo-resources are not actual Kubernetes resources or Custom Resource Definitions (CRDs), they do not exist as objects on the cluster. You cannot create, list, or inspect pseudo-resources by using the Kubernetes API or command-line tools such as **kubectl**.

Access is granted through standard Kubernetes **Role**, **ClusterRole**, **RoleBinding** and **ClusterRoleBinding** objects.

The workspace name serves as the Kubernetes namespace for access checks. The MLflow API access is project-scoped.

The following table outlines primary pseudo-resources:

Table 2.4. MLflow pseudo-resources

Pseudo-resource	Controls access to
experiments	Experiments, runs, traces, artifacts, logged models, scorers, and related tracking operations.
registeredmodels	Registered models, model versions, and prompts.
datasets	Evaluation datasets and related data set operations.

+ .Role assignments for MLflow API usage

The Red Hat OpenShift AI admin, edit, and view roles include the permissions required for MLflow API authorization checks through aggregate **ClusterRoles**. These roles allow users to interact with MLflow components within their assigned namespaces.

Workloads or agents that require MLflow API access without using broad human-facing roles can use built-in integration roles. For example, a dedicated mlflow-integration **ClusterRole** is available for service accounts that need to perform automated tracking or model registry tasks.

+ .Resource-name granularity The MLflow Kubernetes authorization plugin allows you to assign permissions with **resourceName** granularity. For example, you can restrict an agent so that it sends traces to only a specific experiment by defining the following attributes in the RBAC policy:

- **resourceName**: The name of the specific experiment
- **resource**: **experiments**
- **apiGroup**: **mlflow.kubeflow.org**
- **verbs**: **get**, **list**, **update**

CHAPTER 3. INSTALL AND AUTHENTICATE THE MLFLOW SDK

Install the MLflow SDK and configure authentication for your OpenShift cluster to track machine learning experiments in Red Hat OpenShift AI.

Prerequisites

- You have access to a OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) and are able to access MLflow in your OpenShift AI project or MLflow workspace.
- For the automated authentication method, you have a service account with the appropriate permissions or are running locally.

Procedure

1. Install the MLflow SDK:

```
pip install "mlflow[kubernetes]>=3.11"
```

1. Authenticate the SDK by using one of the following methods:

   a. To use automated authentication, enable the **kubernetes-namespaced** client-side authentication plugin.

   NOTE

   This plugin reads credentials from the mounted service account token when running in a pod, or from the active **kubeconfig** context when running on a workstation. The MLflow workspace is automatically configured by reading either the service account's namespace or active **kubeconfig** namespace.

   Enter the following commands:

   ```
   export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
   export MLFLOW_TRACKING_AUTH=kubernetes-namespaced
   ```

   b. To authenticate manually, export your tracking token and project workspace as environment variables. Enter the following commands:

   ```
   export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
   export MLFLOW_TRACKING_TOKEN="$(oc whoami --show-token)"
   export MLFLOW_WORKSPACE="<project-name>"
   ```

   IMPORTANT

   This configuration example is not recommended for production environments.

2. If your OpenShift cluster does not use trusted TLS certificates, enter the following command to disable TLS verification:

```
export MLFLOW_TRACKING_INSECURE_TLS=true
```

Verification

Use the following command to verify connectivity **python -c "import mlflow; print(mlflow.list_workspaces())"** This command lists the workspaces you can access on the OpenShift cluster.

3.1. CONFIGURING THE MLFLOW SDK FOR A LOCAL WORKSTATION

When you run the MLflow SDK on a local workstation, the authentication plugin uses the active kubeconfig context. The plugin uses the namespace from the kubeconfig context as the workspace and resolves the authentication token from your kubeconfig credentials, including exec-based authentication providers used by **oc login** command.

+ .Prerequisites

- You have installed the MLflow SDK.
- You have an active kubeconfig context.

Procedure

+ . Set the following environment variables to configure the tracking URI and authentication method:

+

```
export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
export MLFLOW_TRACKING_AUTH=kubernetes-namespaced
```

1. Optional: If you prefer to set the token and workspace manually, export the following variables:

```
export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
export MLFLOW_TRACKING_TOKEN="$(oc whoami --show-token)"
export MLFLOW_WORKSPACE="<project-name>"
```

3.2. CONFIGURING MLFLOW SDK ENVIRONMENT VARIABLES FOR PODS

+ When you run a pod, the Kubernetes authentication plugin uses the mounted service account token and namespace to automatically set the workspace.

You do not have to call **mlflow.set_workspace()** when you enable the authentication plugin. The plugin derives the workspace from the pod's service account namespace. You can override the workspace explicitly if you need to target a different project, provided that your service account has the necessary RBAC permissions in that project.

Prerequisites

- You have installed the MLflow SDK.

Procedure

1. Set the following environment variables in your pod configuration:

```
export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
export MLFLOW_TRACKING_AUTH=kubernetes-namespaced
```

Verification

1. Run a Python script using the following code to confirm that the MLflow SDK successfully connects to the tracking server and logs data:

```
import mlflow

mlflow.set_experiment("demo-experiment")

with mlflow.start_run():
    mlflow.log_param("framework", "pytorch")
    mlflow.log_metric("accuracy", 0.95)
```

3.3. UPSTREAM MLFLOW SDK REFERENCE

Additional resources

For more information about the upstream MLflow SDK, see the following resources:

- MLflow tracking
- MLflow Tracking API reference
- MLflow Model Registry
- Getting started with MLflow workspaces
- MLflow workspace providers
- MLflow 3 migration guide
- MLflow Python API reference

3.4. MLFLOW SDK TROUBLESHOOTING REFERENCE

If you encounter errors when working with experiments or artifacts in the MLflow SDK, use the following information to resolve common issues and error messages.

If your issue is not described here, contact Red Hat support.

Common issues

403 or permission denied

Problem

The active project is missing the required role-based access control (RBAC) permissions.

Resolution

Verify that your user or service account has the necessary role binding in the active project.

Workspace not found

Problem

The SDK cannot locate the workspace because the project name is incorrect, the namespace is filtered, or no workspace was selected.

Resolution

Verify that your project name is correct and that the namespace is not restricted. Ensure that you have selected a workspace in your MLflow environment settings by using one of the following methods: * The **MLFLOW_TRACKING_AUTH=kubernetes-namespaced** environment variable. * The **MLFLOW_WORKSPACE=<workspace_name>** environment variable. * The **mlflow.set_workspace("team-a")** Python function.

Artifact override is not applied

Problem

The **MLflowConfig** resource is missing, has the wrong name, or exists in the wrong project. Alternatively, the associated secret does not exist.

Resolution

Ensure that an **MLflowConfig** resource exists and is named **mlflow**. Verify that the **mlflow-artifact-connection** secret is present in the namespace.

Kubernetes-namespaced authentication plugin cannot resolve credentials

Problem

The authentication plugin is missing a service account token (when running in-cluster) or an active **kubeconfig** context (when running locally).

Resolution

If running in-cluster, ensure the service account has a valid token. If running locally, verify that your **kubeconfig** context is active and points to the correct cluster and project by running **oc project**.

Artifact writes go to the default storage location

Problem

The **MLflowConfig** resource does not exist in the active project, or the **artifactRootSecret** is invalid.

Resolution

Create the **MLflowConfig** resource in your active project and verify that the **artifactRootSecret** contains the correct connection credentials.

JSON decode error: Expecting value

Problem

The Red Hat OpenShift AI authentication token is missing or invalid. Consequently, OpenShift AI receives an HTML response from MLflow instead of the expected JSON, causing OpenShift AI to prompt for a login.

Resolution

Log in to the OpenShift cluster by running the **oc login** command. Ensure that your environment uses the **MLFLOW_TRACKING_AUTH=kubernetes-namespaced** environment variable to authenticate requests.

3.5. MLFLOW VERSION COMPATIBILITY

The following information describes the compatible versions of MLflow and Red Hat OpenShift AI 3.4 GA.

Table 3.1. MLflow version compatibility and configuration

Item	Description
Deployed MLflow server version	**3.10.1**
Required MLflow SDK version	3.11 or later
Authentication plugin name	**kubernetes-namespaced**
Environment variable	**MLFLOW_TRACKING_AUTH=kubernetes-namespaced**

The following command installs the compatible version of the MLflow SDK:

+

```
pip install "mlflow[kubernetes]>=3.11"
```

+

NOTE

MLflow SDK version 3.11 and later includes the **kubernetes-namespaced** authentication plugin by default.

3.6. MLFLOW STORAGE AND DATABASE COMPATIBILITY

The following table lists the MLflow storage and database configurations. The configuration settings vary according to your environment, for example production, development, or testing.

Table 3.2. Supported storage and database options

Storage area	Supported options
Artifact storage	S3 compatible object storage for production. File system for development and testing.
Database	PostgreSQL for production. SQLite for development and testing.
Artifact repository plugins	S3 and file.

3.7. TRACKING EXPERIMENTS WITH MLFLOW SDK

Use the MLflow software development kit (SDK) to log and track machine learning experiments. With the SDK, you can record parameters, metrics, and artifacts to a centralized tracking server for later analysis.

Prerequisites

- You have installed the MLflow SDK version 3.11 or later.
- You have access to a Red Hat OpenShift AI data science project that has MLflow permissions configured.
- You have configured a tracking URI and authentication. For more information, see MLflow Kubernetes Authentication. .Procedure

1. In your notebook, log experiments, parameters, metrics, and artifacts by using the MLflow SDK:
   When the kubernetes-namespaced authentication plugin is configured, the tracking URI and workspace are resolved automatically.

```
import random
import time
import mlflow

mlflow.set_experiment("demo-experiment")

with mlflow.start_run(run_name="demo-run"):
    mlflow.log_param("model_type", "baseline")
    mlflow.log_param("feature_count", 3)
    for step in range(5):
        mlflow.log_metric("accuracy", 0.8 + random.random() * 0.2, step=step)
        mlflow.log_metric("loss", 0.5 - random.random() * 0.2, step=step)
        time.sleep(0.2)
```

2. Optional: To target a project other than the one associated with your credentials, set the workspace explicitly:

```
mlflow.set_workspace("<project-name>")
```

Troubleshooting

If you encounter errors when using the MLflow SDK to manage experiments or artifacts, see Troubleshooting MLflow SDK.

3.8. CONFIGURE PROJECT-SPECIFIC S3 ARTIFACT STORAGE

By default, all projects use the artifact storage configured in the **MLflow** resource. You can override the artifact storage for a specific project by creating an **MLflowConfig** resource and a connection in that project. After you create these resources, MLflow resolves the artifact root from the override for any new experiments and runs that you create in that project. However, MLflow does not serve artifacts when you configure a per-project override. The client accesses the S3 bucket directly, so the client must have valid S3 credentials.

The **MLflowConfig** resource is namespace-scoped and must be named **mlflow**. It points to an S3 compatible object storage connection that holds the credentials and bucket information for the project.

Prerequisites

- You have installed MLflow.
- You have an S3-compatible object storage bucket with credentials.

Procedure

1. In your project, add a connection of type **S3 compatible object storage**. Set the resource name of the connection to **mlflow-artifact-connection**. Provide the following connection details:
   - **Access key**: The S3 access key ID.
   - **Secret key**: The S3 secret access key.
   - **Endpoint**: The S3 compatible endpoint URL.
   - **Region**: The S3 region. Optional if your storage provider does not require it.
   - **Bucket**: The S3 bucket name.

   If you are using the connections API instead of the dashboard, create a **Secret** with the annotation **opendatahub.io/connection-type-protocol: "s3"** and name it **mlflow-artifact-connection**. Setting an **AWS_DEFAULT_REGION** is optional. The required keys are:
   - **AWS_ACCESS_KEY_ID**
   - **AWS_SECRET_ACCESS_KEY**
   - **AWS_S3_BUCKET**
   - **AWS_S3_ENDPOINT**
2. Create an **MLflowConfig** resource named **mlflow** in the same project:

```
apiVersion: mlflow.kubeflow.org/v1
kind: MLflowConfig
metadata:
  name: mlflow
spec:
  artifactRootSecret: mlflow-artifact-connection
  artifactRootPath: mlflow-artifacts
```

artifactRootSecret must be **mlflow-artifact-connection**. The Custom Resource Definition (CRD) enforces this validation.

NOTE

artifactRootPath is an optional relative path that the system appends to the bucket root from the secret. For example, if the bucket is **ds-team-bucket** and **artifactRootPath** is **mlflow-artifacts**, the resolved artifact root becomes **s3://ds-team-bucket/mlflow-artifacts**. The path must be relative, must not use backslashes, and must not contain path traversal such as **..**.

CHAPTER 4. TRACK EXPERIMENTS WITH MLFLOW IN WORKBENCHES

You can track machine learning experiments in workbench notebooks with the MLflow SDK. OpenShift AI provides automatic MLflow SDK configuration for workbenches, which removes the need to manually set tracking URIs, configure authentication, or manage RBAC permissions.

To use MLflow experiment tracking, a cluster administrator must first enable the MLflow Operator component, and then a data scientist can enable and use the integration in individual workbenches.

4.1. MLFLOW WORKBENCH INTEGRATION

You can enable automatic MLflow SDK configuration in your workbenches by annotating notebook resources with the **opendatahub.io/mlflow-instance** annotation. When this annotation is present, OpenShift AI automatically injects MLflow environment variables and provisions the required RBAC permissions, so that you can track experiments without manual SDK configuration.

The MLflow workbench integration removes the need to manually set tracking URIs, configure authentication tokens, or create Kubernetes RBAC resources. When an administrator enables the MLflow Operator component in the **DataScienceCluster** and creates an MLflow custom resource (CR), the platform handles the remaining configuration at the workbench level.

4.1.1. How the integration works

The integration uses two mechanisms in the notebook controller to configure MLflow access:

Environment variable injection

When a workbench notebook resource has the **opendatahub.io/mlflow-instance** annotation, the notebook controller's mutating webhook injects three environment variables into the notebook container before the pod starts:

- **MLFLOW_TRACKING_URI**: The URL of the MLflow tracking server, constructed from the GatewayAPI hostname and the MLflow instance name.
- **MLFLOW_K8S_INTEGRATION**: Set to **true** to enable Kubernetes service account token authentication with the MLflow server.
- **MLFLOW_TRACKING_AUTH**: Set to **kubernetes-namespaced** to configure namespace-scoped authentication.

These variables allow the MLflow Python SDK to connect to the tracking server without additional configuration in your notebook code.

RBAC provisioning

The notebook controller's reconciler creates a namespace-scoped **RoleBinding** named **__<notebook_name>__-mlflow** that grants the workbench service account the permissions defined in the **mlflow-operator-mlflow-integration ClusterRole**. The **RoleBinding** uses a controller owner reference to the notebook resource, so it is automatically deleted when the notebook is deleted.

4.1.2. Configuration methods

You can enable MLflow integration for workbenches in two ways:

Dashboard-managed

If you use the OpenShift AI dashboard with the Dashboard, MLflow, and Workbenches components set to **Managed** in the **DataScienceCluster**, the dashboard automatically adds the **opendatahub.io/mlflow-instance** annotation when you create or update a workbench. No manual configuration is required.

Manual

If you manage workbenches through GitOps, Helm charts, or direct YAML manifests without the OpenShift AI dashboard, you must manually add the **opendatahub.io/mlflow-instance** annotation to the notebook resource.

4.1.3. Annotation lifecycle

The **opendatahub.io/mlflow-instance** annotation controls the MLflow integration for each workbench:

Adding the annotation

You can add the annotation to a stopped notebook or include it when you create a new notebook. The annotation value must be the name of the MLflow instance CR. Environment variables are injected on the next pod start, and the **RoleBinding** is created by the reconciler.

Removing the annotation

You must stop the workbench before removing the annotation. The **RoleBinding** is deleted immediately when the annotation is removed, but environment variables in the running pod persist until restart. This mismatch can cause authentication failures if the workbench continues to send requests to the MLflow tracking server after the **RoleBinding** is removed. A validating webhook checks that the **kubeflow-resource-stopped** annotation is present on the notebook resource before it allows removal of the MLflow annotation. The webhook validates the annotation state, not the actual pod state. If you try to remove the annotation without first stopping the workbench through the dashboard or by setting the stop annotation, the API request is rejected with a webhook error.

4.1.4. Non-blocking behavior

MLflow integration failures do not block workbench admission. If the tracking URI cannot be determined, for example because the GatewayAPI hostname is not yet available, the webhook skips the **MLFLOW_TRACKING_URI** injection but still allows the notebook to start. In this case, **MLFLOW_K8S_INTEGRATION** and **MLFLOW_TRACKING_AUTH** are still injected, which creates a degraded state in which the MLflow SDK is configured for Kubernetes authentication but has no tracking server to connect to. If MLflow operations such as **mlflow.set_experiment()** fail with a **ConnectionError**, verify that the **MLFLOW_TRACKING_URI** environment variable is set in your workbench pod.

Similarly, if the **mlflow-operator-mlflow-integration ClusterRole** does not yet exist, the reconciler requeues the **RoleBinding** creation with a warning event every 30 seconds until the **ClusterRole** becomes available.

4.1.5. Limitations

Artifact-serving configuration with S3 connection types is not supported. You can log parameters, metrics, and tags, but **mlflow.log_artifact()** functionality that relies on an S3-backed artifact store requires additional configuration that is outside the scope of the automatic integration.

4.2. ENABLE THE MLFLOW OPERATOR COMPONENT

You can enable the MLflow operator as a managed component in the **DataScienceCluster** so that the MLflow tracking server and workbench integration are available on the platform. When the MLflow operator component is enabled, workbench notebooks can use automatic MLflow SDK configuration.

Prerequisites

- The Red Hat OpenShift AI Operator is installed on your OpenShift cluster.
- A **DataScienceCluster** object exists.
- You have cluster administrator privileges.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Log in to your OpenShift cluster as a cluster administrator:

```
$ oc login --token=__<token>__ --server=__<openshift_cluster_url>__
```

2. Identify the name of the **DataScienceCluster** object:

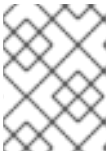
```
$ oc get datasciencecluster
```

3. Edit the **DataScienceCluster** object to enable the MLflow operator component:

```
$ oc edit datasciencecluster __<dsc_name>__
```

4. In the **spec.components** section, add or update the **mlflowoperator** field and set **managementState** to **Managed**:

```
spec:
  components:
    # ...
    mlflowoperator:
      managementState: Managed
    # ...
```



NOTE

The default **managementState** for the **mlflowoperator** component is **Removed**. You must explicitly set it to **Managed** to enable MLflow on the platform.

5. Save and close the editor.
When you set **mlflowoperator** to **Managed**, the Red Hat OpenShift AI Operator automatically configures the notebook controller with **MLFLOW_ENABLED=true** and sets the appropriate **GATEWAY_URL** for the cluster.

6. Create an MLflow CR to deploy a tracking server instance. The MLflow CR structure and namespace requirements depend on the MLflow operator version. For more information, consult the MLflow operator documentation for the CR specification.

Verification

Verification

1. Confirm that the MLflow operator pod is running:

```
$ oc get pods -n redhat-ods-applications -l app=mlflow-operator --field-selector
status.phase=Running
```

The output shows one or more MLflow operator pods in **Running** status. If the command returns no results, verify that the **mlflowoperator** component **managementState** is set to **Managed** and wait for the operator pods to start.

2. Confirm that the **mlflowoperator** component status is **true** in the **DataScienceCluster**:

```
$ oc get datasciencecluster __<dsc_name>__ -o
jsonpath='{.status.installedComponents.mlflowoperator}'
```

The expected output is **true**.

4.2.1. MLflow dashboard feature flag deprecation

Starting with Red Hat OpenShift AI 3.4, the **mlflow** field in the **OdhDashboardConfig** CR is deprecated. MLflow availability in the dashboard is now determined by the **mlflowoperator** component state in the **DataScienceCluster**, and the dashboard feature flag is no longer required.

4.2.1.1. Deprecated field

The **mlflow** field in **spec.dashboardConfig** of the **OdhDashboardConfig** CR is deprecated and no longer controls MLflow visibility in the dashboard. You do not need to set this field to enable MLflow. The field has no effect on MLflow functionality.

4.2.1.2. Current behavior

MLflow features in the OpenShift AI dashboard are automatically available when the **mlflowoperator** component is set to **Managed** in the **DataScienceCluster**. No additional dashboard configuration is required.

4.2.1.3. mlflowPipelines field

The **mlflowPipelines** field in the **OdhDashboardConfig** CR is **not** deprecated and remains active. This field controls whether pipeline run tables display the MLflow experiment column. Do not confuse the deprecated **mlflow** field with the active **mlflowPipelines** field.

Table 4.1. Dashboard feature flag status

Field	Status	Description
spec.dashboardConfig.mlflow	Deprecated	In earlier releases, this field controlled MLflow visibility in the dashboard. No longer required; MLflow is enabled automatically by the **mlflowoperator** component.
spec.dashboardConfig.mlflowPipelines	Active	Controls whether the MLflow experiment column is displayed in pipeline run tables.

Field	Status	Description

4.3. ENABLE MLFLOW INTEGRATION FOR A WORKBENCH

You can enable automatic MLflow SDK configuration for a workbench by adding the **opendatahub.io/mlflow-instance** annotation to the notebook resource. When the workbench starts, the platform injects MLflow environment variables and provisions the required RBAC resources.

When you create or update a workbench, the OpenShift AI dashboard automatically adds the MLflow annotation. For this to occur, the following components in the **DataScienceCluster** object must be set to **Managed**:

- Dashboard
- MLflow
- Workbenches

The following procedure is for environments where you manage workbenches through GitOps, Helm charts, or direct YAML manifests without the OpenShift AI dashboard.

Prerequisites

- A cluster administrator has enabled the MLflow Operator component in the **DataScienceCluster** object. For more information, see Enable the MLflow Operator component.
- An MLflow CR has been created in the same namespace as the workbench.
- The workbench is stopped.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Log in to your OpenShift cluster:

```
$ oc login --token=__<token>__ --server=__<openshift_cluster_url>__
```

2. Add the **opendatahub.io/mlflow-instance** annotation to the notebook resource.
   In the following command, replace **__<notebook_name>__** with the name of your workbench notebook and **__<mlflow_instance_name>__** with the name of your MLflow instance CR:

```
$ oc annotate notebook -n __<namespace>__ __<notebook_name>__
opendatahub.io/mlflow-instance=__<mlflow_instance_name>__
```

   If the MLflow instance name is **mlflow**, the tracking URI path is **/mlflow**. If the instance name is different, the path is **/mlflow-__<mlflow_instance_name>__**.

Alternatively, you can add the annotation directly in the notebook resource YAML:

```
apiVersion: kubeflow.org/v1
kind: Notebook
metadata:
  name: __<notebook_name>__
  namespace: __<namespace>__
  annotations:
    opendatahub.io/mlflow-instance: "__<mlflow_instance_name>__"
spec:
  # ...
```

3. Start or restart the workbench so that the environment variables are injected into the new pod.

Verification

1. Verify that the MLflow environment variables are present in the running workbench pod. First, find the pod name for your notebook:

```
$ oc get pods -n __<namespace>__ -l notebook-name=__<notebook_name>__
```

Then verify that the MLflow environment variables are present, where **__<notebook_pod_name>__** is the pod name from the previous command and **__<notebook_name>__** is the notebook resource name:

```
$ oc exec -n __<namespace>__ __<notebook_pod_name>__ -c __<notebook_name>__ --
env | grep MLFLOW
```

The expected output shows the following environment variables:

```
MLFLOW_TRACKING_URI=https://<gateway_hostname>/mlflow
MLFLOW_K8S_INTEGRATION=true
MLFLOW_TRACKING_AUTH=kubernetes-namespaced
```

The **<gateway_hostname>** value is cluster-specific and matches the GatewayAPI public endpoint for your cluster.

2. Verify that the MLflow **RoleBinding** exists in the namespace:

```
$ oc get rolebinding -n __<namespace>__ __<notebook_name>__-mlflow
```

The output shows a **RoleBinding** referencing the **mlflow-operator-mlflow-integration ClusterRole**.

4.4. USE THE MLFLOW SDK IN A WORKBENCH NOTEBOOK

After MLflow integration is enabled for your workbench, you can use the MLflow Python SDK to create experiments, log parameters and metrics, and track runs. The required environment variables are already configured, so the SDK connects to the tracking server automatically.

Prerequisites

- A cluster administrator has enabled the MLflow operator component in the **DataScienceCluster**. For more information, see Enable the MLflow operator component .
- An MLflow CR has been created in the namespace.
- The workbench notebook resource has the **opendatahub.io/mlflow-instance** annotation. For more information, see Enable MLflow integration for a workbench .
- The workbench is started and the MLflow environment variables are injected.
- The **mlflow** Python package is installed in the workbench. Starting with OpenShift AI 3.4, all workbench images except the minimal image include the MLflow SDK. For older images or custom images, install the package by running **pip install mlflow** in a notebook cell.

NOTE

If you need only basic tracking functionality, you can install the **mlflow-skinny** package instead for a smaller footprint. The **mlflow-skinny** package contains the core tracking and logging features but does not include model serving integrations or the full MLflow CLI.

Procedure

1. Open a notebook in the workbench.
2. Verify that the MLflow environment variables are available:

```
import os
print(os.environ.get("MLFLOW_TRACKING_URI"))
print(os.environ.get("MLFLOW_K8S_INTEGRATION"))
print(os.environ.get("MLFLOW_TRACKING_AUTH"))
```

   All three variables display their configured values.

3. Import the MLflow SDK and set an experiment:

```
import mlflow

mlflow.set_experiment("my-experiment")
```

   The **set_experiment()** call returns an **Experiment** object and prints the experiment ID. If you see a **ConnectionError**, verify that the **MLFLOW_TRACKING_URI** environment variable is set correctly by re-running the check in step 2.

4. Start a run and log parameters and metrics:

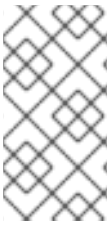
```
with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.01)
    mlflow.log_param("epochs", 10)
    mlflow.log_metric("accuracy", 0.95)
    mlflow.log_metric("loss", 0.05)
```

   You can log any number of parameters, metrics, and tags within a run. If **start_run()** raises a **ConnectionError** or an authentication or authorization error, verify that the MLflow tracking server is running and that the workbench **RoleBinding** exists.

5. View the experiment results in the MLflow UI, which is accessible from the OpenShift AI dashboard.

Verification

- Confirm that the experiment and run are displayed in the MLflow UI. Navigate to the MLflow tracking server URL in your browser or through the OpenShift AI dashboard and verify that the logged parameters and metrics are visible.



NOTE

Artifact-serving configuration with S3 connection types is not supported. You can log parameters, metrics, and tags, but **mlflow.log_artifact()** functionality that relies on an S3-backed artifact store requires additional configuration. For more information, see MLflow workbench integration .

4.5. DISABLE MLFLOW INTEGRATION FOR A WORKBENCH

You can disable MLflow integration for a workbench by removing the **opendatahub.io/mlflow-instance** annotation from the notebook resource. You must stop the workbench before removing the annotation.

IMPORTANT

When you remove the annotation, the platform deletes the associated **RoleBinding** resource immediately, but environment variables in the running pod persist until the pod restarts. If the workbench is running when you remove the annotation, the MLflow SDK continues to send requests to the tracking server without a valid **RoleBinding** resource in place, which causes authentication failures.

Prerequisites

- Your workbench has the **opendatahub.io/mlflow-instance** annotation.
- The workbench is stopped. The validating webhook requires the **kubeflow-resource-stopped** annotation to be present on the notebook resource before it allows removal of the MLflow annotation.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Log in to your OpenShift cluster:

```
$ oc login --token=__<token>__ --server=__<openshift_cluster_url>__
```

2. Verify that the workbench pod has stopped:

```
$ oc get pods -n __<namespace>__ -l notebook-name=__<notebook_name>__
```

   The command produces no output when the pod has stopped.

3. Remove the **opendatahub.io/mlflow-instance** annotation from the notebook resource.

In the following command, replace **__<notebook_name>__** with the name of your workbench notebook:

```
$ oc annotate notebook -n __<namespace>__ __<notebook_name>__
opendatahub.io/mlflow-instance-
```

The trailing dash (**-**) removes the annotation.

Verification

1. Start the workbench and verify that the MLflow environment variables are no longer present. First, find the pod name for your notebook:

```
$ oc get pods -n __<namespace>__ -l notebook-name=__<notebook_name>__
```

Then verify that the MLflow environment variables are no longer present, where **__<notebook_pod_name>__** is the pod name from the previous command and **__<notebook_name>__** is the notebook resource name:

```
$ oc exec -n __<namespace>__ __<notebook_pod_name>__ -c __<notebook_name>__ --
env | grep MLFLOW
```

The command produces no output, confirming that the MLflow variables have been removed.

2. Verify that the MLflow **RoleBinding** resource has been removed:

```
$ oc get rolebinding -n __<namespace>__ __<notebook_name>__-mlflow
```

The expected output is:

```
Error from server (NotFound): rolebindings.rbac.authorization.k8s.io
"__<notebook_name>__-mlflow" not found
```

Troubleshooting

If the annotation removal is rejected with a webhook error, see Resolve MLflow annotation removal rejection.

4.6. RESOLVE MLFLOW ANNOTATION REMOVAL REJECTION

If you try to remove the **opendatahub.io/mlflow-instance** annotation from a running workbench, the API request is rejected by a validating webhook. You can resolve this issue by stopping the workbench before removing the annotation.

The validating webhook checks that the **kubeflow-resource-stopped** annotation is present on the notebook resource before it allows removal of the MLflow annotation. The webhook validates the annotation state, not the actual pod state. If the pod was stopped without setting the stop annotation, for example by deleting the pod directly, the webhook still rejects the annotation removal.

Prerequisites

- The workbench has the **opendatahub.io/mlflow-instance** annotation.

- You received a webhook error when trying to remove the annotation.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Stop the workbench. You can stop the workbench from the OpenShift AI dashboard or by applying the stop annotation:

```
$ oc annotate notebook -n __<namespace>__ __<notebook_name>__ kubeflow-resource-stopped=true
```

   The value of the **kubeflow-resource-stopped** annotation is not significant for this purpose. The validating webhook checks only for the presence of the annotation key.

2. Wait for the notebook pod to stop fully:

```
$ oc get pods -n __<namespace>__ -l notebook-name=__<notebook_name>__
```

   The command produces no output when the pod has stopped.

3. Remove the MLflow annotation:

```
$ oc annotate notebook -n __<namespace>__ __<notebook_name>__ opendatahub.io/mlflow-instance-
```

Verification

- Confirm that the annotation has been removed:

```
$ oc get notebook -n __<namespace>__ __<notebook_name>__ -o jsonpath='{.metadata.annotations.opendatahub\.io/mlflow-instance}'
```

  The command produces no output, confirming the annotation is removed.

4.7. MLFLOW WORKBENCH ENVIRONMENT VARIABLES AND ANNOTATIONS

When MLflow integration is enabled for a workbench, the notebook controller injects environment variables and creates RBAC resources automatically. You can use this reference to understand the annotation, environment variables, and **RoleBinding** that the platform manages on your behalf.

4.7.1. Annotation

Table 4.2. MLflow instance annotation

Annotation	Description
opendatahub.io/mlflow-instance	Specifies the name of the MLflow instance CR to connect to. When this annotation is present and non-empty on a notebook resource, the notebook controller enables MLflow integration for the workbench. The annotation value determines the path segment in the tracking URI.

Table 4.3. Annotation value to tracking URI path mapping

Annotation value	Tracking URI path
mlflow	**/mlflow**
__<custom_instance_name>__	**/mlflow-__<custom_instance_name>__**

4.7.2. Environment variables

The following environment variables are injected into the notebook container when the **opendatahub.io/mlflow-instance** annotation is present.

Table 4.4. MLflow environment variables

Variable	Value	Description
MLFLOW_TRACKING_URI	**\https://__<gateway_hostname>__/mlflow** or **\https://__<gateway_hostname>__/mlflow-__<instance_name>__**	Specifies the URL of the MLflow tracking server. The hostname is derived from the GatewayAPI public endpoint. If the instance name is **mlflow**, the path is **/mlflow**. If the instance name differs from **mlflow**, the path is **/mlflow-__<instance_name>__**.
MLFLOW_K8S_INTEGRATION	**true**	Specifies that the MLflow client uses Kubernetes service account token authentication.
MLFLOW_TRACKING_AUTH	**kubernetes-namespaced**	Specifies the authentication method for the MLflow client. This value configures namespace-scoped authentication using the workbench service account.

4.7.3. RoleBinding

The notebook controller creates a namespace-scoped **RoleBinding** for each annotated workbench.

Table 4.5. MLflow RoleBinding details

Property	Value
Name	**__<notebook_name>__-mlflow**
Namespace	Same as the notebook resource namespace
Subject	**ServiceAccount** with the name of the notebook, in the notebook namespace
RoleRef	**ClusterRole** named **mlflow-operator-mlflow-integration**

Property	Value
Controller owner reference	Set to the notebook resource, so the **RoleBinding** is automatically deleted when the notebook is deleted
Labels	**notebook-name: __<notebook_name>__**

4.7.4. ClusterRole availability

The **RoleBinding** references the **mlflow-operator-mlflow-integration ClusterRole**, which is created by the MLflow operator. If this **ClusterRole** does not yet exist when the reconciler attempts to create the **RoleBinding**, the reconciler requeues the request with a warning event every 30 seconds until the **ClusterRole** becomes available. These events are generated only when a notebook resource has the **opendatahub.io/mlflow-instance** annotation but the **ClusterRole** does not yet exist. If no annotated notebooks exist, no events are generated. You can observe these events by running the following command, where **__<namespace>__** is the namespace containing your workbench notebook:

```
$ oc get events -n __<namespace>__ --field-selector reason=MLflowClusterRolePending
```

4.7.5. Controller environment variables

The Red Hat OpenShift AI Operator automatically configures the following environment variables on the notebook controller when the **mlflowoperator** component is set to **Managed** in the **DataScienceCluster**. You do not need to set these variables manually.

Table 4.6. Controller environment variables

Variable	Description
MLFLOW_ENABLED	Set to **true** when the **mlflowoperator** component is **Managed** in the **DataScienceCluster**. Enables the notebook controller to process MLflow annotations on workbench resources.
GATEWAY_URL	Set based on the cluster's Gateway configuration. The notebook controller uses this value to construct the **MLFLOW_TRACKING_URI** for workbench pods.